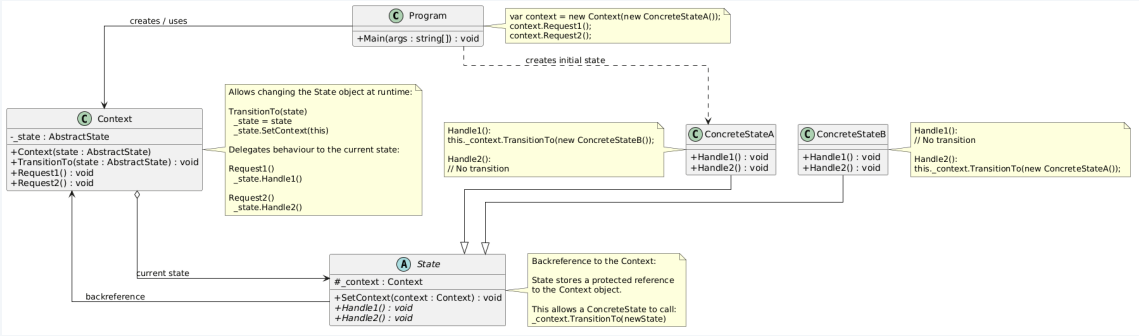
The State Pattern

The State Pattern

- **Intent:** Allow an object to alter its behaviour when its internal state changes. It will appear as if the object changed its class.
- **Problem/ Motivation:** At any given moment, there's a finite number of states in which a program can be. Within each state, the program behaves differently, and the program can be switched from one state to another instantaneously. Example: Think about an article that you want to publish in a conference proceedings. An article can be in one of these states: Draft, Submitted, Under Review, Accepted, Rejected and Published.
- **Solution:** Create classes for all possible states of an object and extract all state-specific behaviors into these classes, i.e., take each state in your design and encapsulate it in a class that implements a State interface.

Structure


creates / uses
Program
+Main(args : string[]) : void
var context = new Context(new ConcreteStateA());
context.Request1();
context.Request2();
creates initial state
Context
-_state : AbstractState
+Context(state : AbstractState)
+TransitionTo(state : AbstractState) : void
+Request1() : void
+Request2() : void
Allows changing the State object at runtime:
TransitionTo(state)
_state = state
_state.SetContext(this)
Delegates behaviour to the current state:
Request1()
_state.Handle1()
Request2()
_state.Handle2()
Handle1():
this._context.TransitionTo(new ConcreteStateB());
Handle2():
// No transition
ConcreteStateA
+Handle1() : void
+Handle2() : void
ConcreteStateB
+Handle1() : void
+Handle2() : void
Handle1():
// No transition
Handle2():
this._context.TransitionTo(new ConcreteStateA());
current state
backreference
State
#_context : Context
+SetContext(context : Context) : void
+Handle1() : void
+Handle2() : void
Backreference to the Context:
State stores a protected reference to the Context object.
This allows a ConcreteState to call:
_context.TransitionTo(newState)

Participants

- **State**
  - Defines an interface common to all ConcreteStates and declares the behaviours applicable to all ConcreteStates
  - ConcreteStates implement the same interface so they are interchangeable
  - May Stores a backreference to the Context object, allowing the state to fetch any required info from the Context object, as well as initiate state transitions.
- **ConcreteState**
  - Each ConcreteState implements a behaviour associated with a state of the Context, i.e., they provide their own implementations for the state-specific methods.
- **Context**
  - Defines the interface of interest to clients.
  - Maintains an instance of a ConcreteState that defines the current state and delegates to it all the state-specific work.
  - Communicates with the state object via the State interface.

C# Code Example – Context

```
class Context
{
    // A reference to the current state of the Context.
    private State _state = null!;

    public Context(State state)
    {
        TransitionTo(state);
    }

    // The Context allows changing the State object at runtime.
    public void TransitionTo(State state)
    {
        _state = state;
        _state.SetContext(this);
    }

    // The Context delegates behaviour to the current State object.
    public void Request1()
    {
        _state.Handle1();
    }

    public void Request2()
    {
        _state.Handle2();
    }
}
```

C# Code Example – Abstract State

```
abstract class State
{
    protected Context _context;

    public void SetContext(Context context)
    {
        this._context = context;
    }

    public abstract void Handle1();

    public abstract void Handle2();
}
```

- The protected `_context` field is a backreference to the `Context`.
- Concrete states use this backreference to trigger state transitions.

C# Code Example – Concrete State A

```
class ConcreteStateA : State
{
    public override void Handle1()
    {
        _context.TransitionTo(new ConcreteStateB());
    }

    public override void Handle2()
    {
        // No transition.
    }
}
```

- Changes the context to `ConcreteStateB`.

Group Exercise – State Pattern

- Vending machine: Discuss how you would implement the State Pattern for a beverage vending machine. What states would the machine be in?
- Document the pattern participants using the next slide.

Documenting the State Pattern – SENG301 Style

GoF Term	Our Design
Context	
State	
ConcreteState	

Table: State Pattern

Pros and Cons

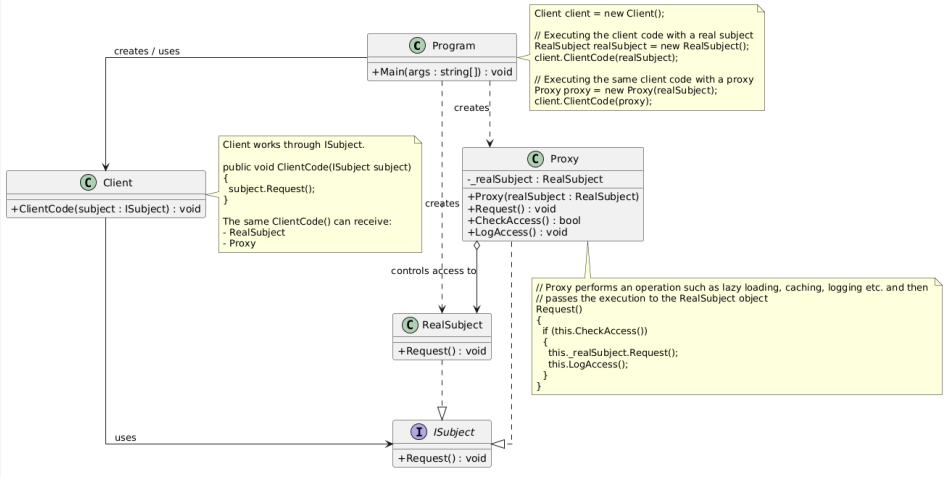
- **Pros**
  - Since all state-specific code lives in a separate State class(s), new states and transitions can be added easily by defining new state classes.
  - Adheres to Open/Closed Principle since we can introduce new states without changing existing state classes or the context
  - Adheres to the Single Responsibility Principle since code related to particular states are organized into separate classes Introducing separate objects for different states makes the transition more explicit
  - Avoids large conditional statements in the Context
- **Cons**
  - Creates a lot of classes and is less compact compared to a single class.
  - If your program has only a few states and the states rarely change, implementing the pattern could be overkill.

The Proxy Pattern

The Proxy Pattern

- **Intent:** Provide a surrogate or placeholder or substitute for another object (e.g., a service), to control access to it, i.e., not allow an object to directly access (or interact with) another object (the service).
- **Problem/ Motivation:** Create a representative object that controls access to another object, which may be remote, expensive to create or in need of securing.
  Real-life example of a proxy: A credit card is a proxy for a bank account or cash. It eliminates the need to carry cash.
- **Solution:** Create a proxy class with the same interface as an original service object.

Structure


Client client = new Client();
// Executing the client code with a real subject
RealSubject realSubject = new RealSubject();
client.ClientCode(realSubject);
// Executing the same client code with a proxy
Proxy proxy = new Proxy(realSubject);
client.ClientCode(proxy);
creates / uses
Program
+Main(args : string[]) : void
creates
Client works through ISubject.
public void ClientCode(ISubject subject)
{
subject.Request();
}
The same ClientCode() can receive:
- RealSubject
- Proxy
Client
+ClientCode(subject : ISubject) : void
creates
Proxy
-_realSubject : RealSubject
+Proxy(realSubject : RealSubject)
+Request() : void
+CheckAccess() : bool
+LogAccess() : void
controls access to
// Proxy performs an operation such as lazy loading, caching, logging etc. and then
// passes the execution to the RealSubject object
Request()
{
if (this.CheckAccess())
{
this._realSubject.Request();
this.LogAccess();
}
}
RealSubject
+Request() : void
uses
ISubject
+Request() : void

Participants

- **Proxy**
  - Has a reference field that points to a service object.
  - Controls access to the RealSubject and sometimes may be responsible for creating and deleting it.
  - Once the proxy finishes its processing (e.g., caching), it passes the request to the service object.
- **Subject (ServiceInterface)**
  - Defines the common interface for RealSubject and Proxy, i.e., both the Proxy and the RealSubject implement the same Subject interface
  - Allows any Client to treat the Proxy like a RealSubject
- **RealSubject (Service)**
  - Defines the real object that the Proxy represents or controls access to.
  - Does most of the real work, i.e., provides some useful business logic.
- **Client**
  - Works with all objects (both subjects and proxies) via the Subject interface.

Types/ Uses/ Implementations of Proxy

- **Remote**
  - Provides a local representative for an object living in a different address space, i.e., controls access to a remote object
- **Virtual**
  - Creates expensive objects on demand (or when it is needed), i.e., controls access to a resource that is expensive to create, e.g., an Image proxy
  - Defers the full cost of an object's creation and initialization until we actually need to use it
- **Protection/ Access control**
  - Used when objects could have different access rights – Controls access to the original object based on access rights

Types/ Uses/ Implementations of Proxy contd.

- **Smart reference**
  - A replacement for a direct reference that performs additional actions when an object is accessed, e.g., counting the number of references to a real object, checking that a real object is locked before accessing it
- **Caching**
  - e.g., pre-fetching results from a database
- **Logging**
  - Actions the client does not need to know or care about

Group Exercise – Proxy Pattern

- Image loading: Imagine you want to display an Image in an application. However, the image is a heavyweight one. A virtual proxy can be used to manage the background loading of an image, and while the image is being fully loaded, you can display a message such as ”Loading image, please wait...”.
- Discuss in groups how you may go about your design.
- Use the next slide to document.

Documenting the Proxy Pattern – SENG301 Style

GoF Term	Our Design
Proxy	
Subject	
RealSubject	
Client	

Table: Proxy Pattern

Pros and Cons

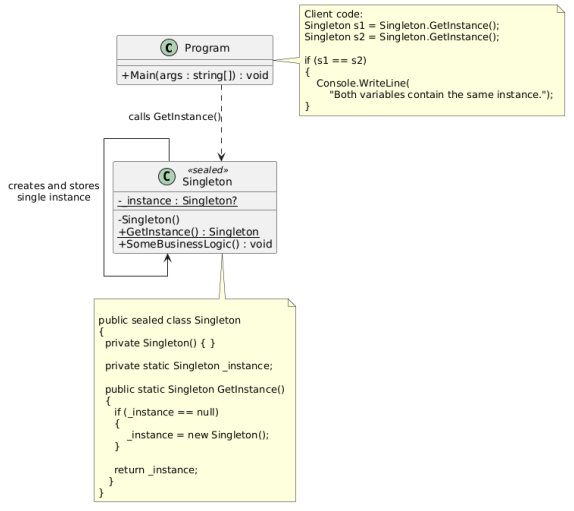
- **Pros**
  - Introduces 'indirection' that has many uses depending on the type of proxy. e.g., a remote proxy can hide the fact that an object resides in a different address space, a virtual proxy can allow the creation of an object on demand, and smart references enable additional housekeeping tasks
  - 'copy-on-write' reduces the cost of copying heavyweight Subjects – using a proxy to postpone the copying process, because copying a large, complex object can be an expensive operation
  - Adheres to the Open/Closed Principle – can introduce new proxies without changing the service or clients.
  - Can control the service object or manage the lifecycle of the service object without clients knowing about it.
  - The proxy works even if the service object isn't ready or is not available.
- **Cons**
  - The response from the service might get delayed.

The Singleton Pattern

The Singleton Pattern

- **Intent:** Ensure a class only has one instance, and provide a global point of access to it
- **Problem/ Motivation:** It is important for some classes to have exactly one instance. The most common reason for using the singleton is to control access to some shared resource, such as a database or a file.
- **Solution:** The class can ensure that no other instance can be created when there is an existing instance and provide access to it.
- The default constructor is private. A static method 'GetInstance' returns the same instance of its own class. – whenever that method is called, the same object is always returned.

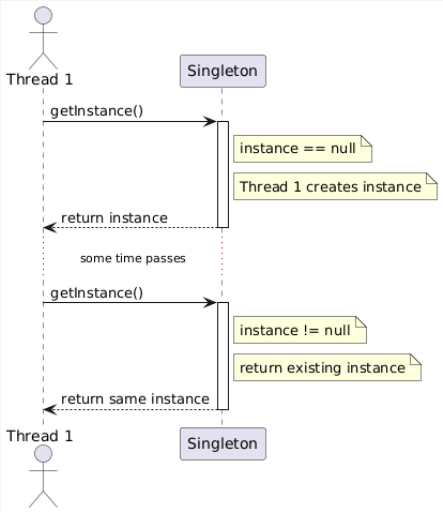
Structure


Client code:
Singleton s1 = Singleton.GetInstance();
Singleton s2 = Singleton.GetInstance();
if (s1 == s2)
{
Console.WriteLine(
"Both variables contain the same instance.");
}
Program
+Main(args : string[]) : void
calls GetInstance()
creates and stores
single instance
«sealed»
Singleton
- instance : Singleton?
-Singleton()
+GetInstance() : Singleton
+SomeBusinessLogic() : void
public sealed class Singleton
{
private Singleton() { }
private static Singleton _instance;
public static Singleton GetInstance()
{
if (_instance == null)
{
_instance = new Singleton();
}
return _instance;
}
}

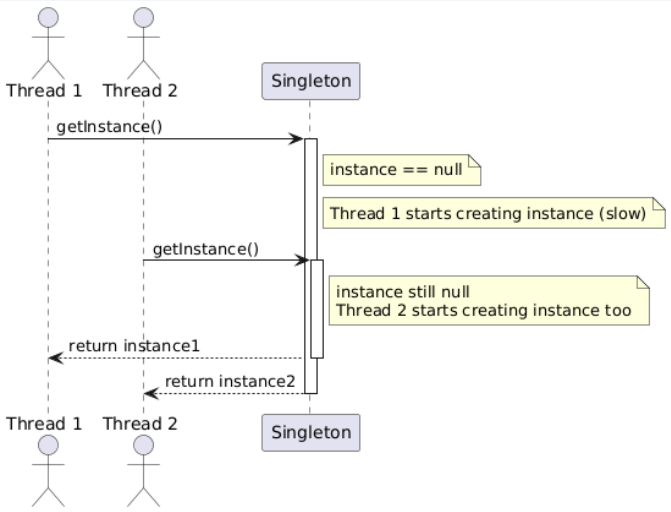
Participants

- **Singleton**
  - Declares the static method GetInstance that returns the same instance of its own class.
  - Constructor is private.
  - Calling the GetInstance method is the only way to obtain an object from Singleton.
- **Client**
  - Client obtains the singleton instance.

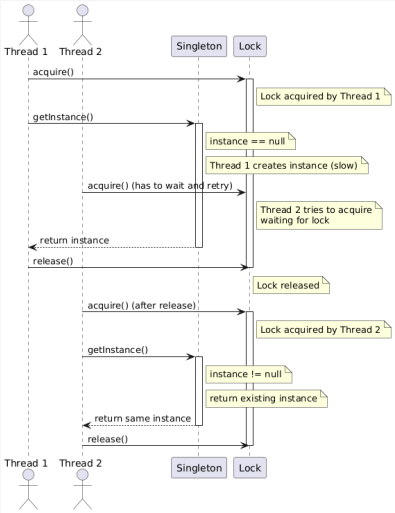
Naïve Singleton (single-threaded)



Naïve Singleton (multi-threaded)



Thread-safe Singleton



Pros and Cons

- **Pros**
  - Ensure that a class has only a single instance
  - Gain global access to that instance
  - Object is initialized only once
- **Cons**
  - Violates the Single Responsibility Principle (two responsibilities)
  - The pattern requires locks to be implemented in a multithreaded environment so that multiple threads don't create a singleton object multiple times
  - Unit testing could be tricky since the constructor of the singleton class is private, and having to override a static method.

The Observer Pattern

Observer Pattern (aka Publish-Subscribe)

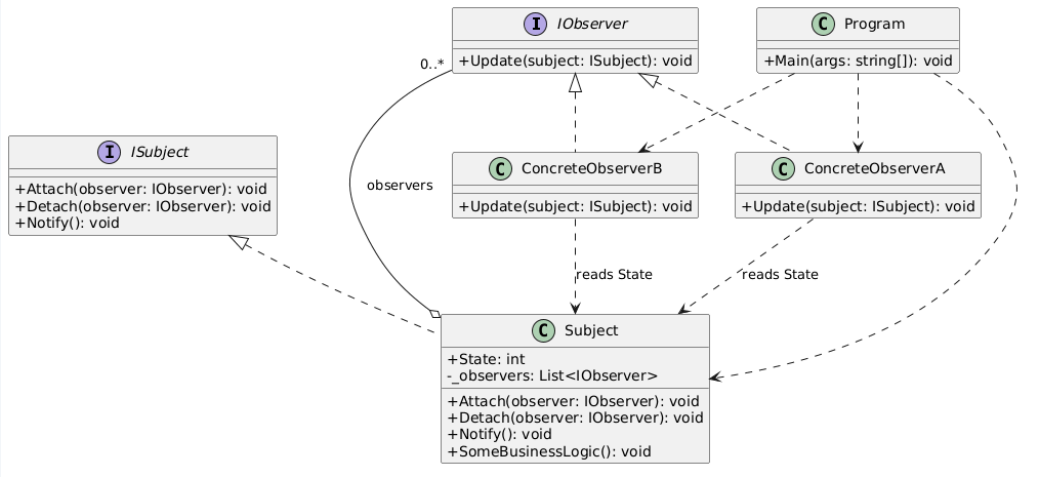
- **Intent:** Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. i.e., this pattern lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.
- **Problem/ Motivation:** Separating concerns into different classes, but keep them in sync. Avoid tight coupling.
  - A newspaper or magazine subscription service with its publisher and subscribers.
  - YouTube channel subscriptions.
  - New iPhone model is out! – Visiting the store vs. sending spam. Either the customer wastes time checking product availability, or the store wastes resources notifying the wrong set of customers.

The Observer Pattern contd.

- **Solution:** Publish-subscribe i.e., an observable Subject and Observers
  - All subscribers implement the same interface, and the publisher communicates with them via that interface.
  - The Observer interface allows the subject to notify observers without depending on their concrete classes.
  - The publisher maintains a list of subscribers and knows what they are interested in.
  - Subscribers can leave the list at any time.

*C# supports Observer-style designs through **events, delegates**, EventHandler<TEventArgs> and the IObservable<T> / IObserver<T>* interfaces.
In Java, implementations of java.util.EventListener.

Structure


IObserver
+Update(subject: ISubject): void
Program
+Main(args: string[]): void
0..*
ISubject
+Attach(observer: IObserver): void
+Detach(observer: IObserver): void
+Notify(): void
observers
ConcreteObserverB
+Update(subject: ISubject): void
ConcreteObserverA
+Update(subject: ISubject): void
reads State
reads State
Subject
+State: int
-_observers: List<IObserver>
+Attach(observer: IObserver): void
+Detach(observer: IObserver): void
+Notify(): void
+SomeBusinessLogic(): void

Participants

- **Subject (Publisher)** – could be an interface if there are multiple concrete subjects (i.e., publishers)
  - Issues events of interest to other objects. These events occur when the Subject (i.e., publisher) changes its state or executes some behaviors.
  - Knows its observers. Any number of Observer objects may observe a Subject. The Subject can register or remove observers.
  - When a new event happens, the publisher goes over the subscription list and calls the update method declared in the Observer interface on each Observer object.
- **Observer (Subscriber)**
  - Defines the notification interface for objects that should be notified of changes in a Subject (i.e., Publisher).
  - In most cases, the interface consists of a single update method. The method may have several parameters that let the publisher pass some event details.
  - A publisher can also pass itself as an argument in the notification method, letting the subscriber fetch any required data directly.

Participants

- **ConcreteObserver (ConcreteSubscriber)**
  - Is registered with a concrete subject to receive updates.
  - Performs some actions in response to notifications issued by the subject/ publisher.
  - Implements the Observer interface to keep its state consistent with the subject's.
- **ConcreteSubject (ConcretePublisher)**
  - Implements the Subject interface.
  - Stores state of interest to ConcreteObserver objects.
  - Sends notification to its observers when its state changes.
- **Client**
  - The Client creates publisher and subscriber objects separately and then registers subscribers for publisher updates.

Documenting the Observer Pattern – SENG301 Style

GoF Term	Our Design
Subject / Publisher	
ConcreteSubject / ConcretePublisher	
Observer / Subscriber	
ConcreteObserver / ConcreteSubscriber	
Attach / Subscribe	
Detach / Unsubscribe	
Notify	
Update	
Client	

Table: Observer Pattern

Pros and Cons

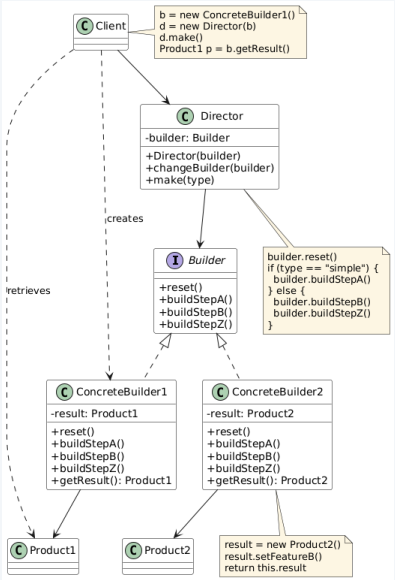
- **Pros**
  - Support for broadcast communication. Meaning, doesn't need to specify the receiver.
  - Subjects and Observers are loosely coupled – minimizes interdependencies between objects.
  - Adheres to the Open/Closed Principle: Can introduce new subscriber classes without having to change the publisher's code.
  - Can establish relations between objects at runtime.
  - Reuse subjects or observers independently.
  - Observers are dependent on the subject for notifications (to update them when the data changes). – cleaner OO design than allowing many objects to control the same data.
- **Cons**
  - Unexpected cascading updates – Small changes in the subject may trigger many unexpected updates in observers and other related objects.
  - Subscribers may be notified in random order. But with a List<T>, they can be notified in registration order.

The Builder Pattern

The Builder Pattern

- **Intent:** Separate the construction of a complex object from its representation so that the same construction process can create different representations.
- **Problem/ Motivation:** Useful when you need to create an object with many possible configuration options. A complex object may require a step-by-step initialization of many fields and nested objects. You want the algorithm for creating complex objects to be independent of the parts that make up the object and how they are assembled.
- **Solution:** Extract the object construction code out of its own class and move it to separate objects called 'builders'. i.e., encapsulates the way the complex object is constructed.

Structure


Client
b = new ConcreteBuilder1()
d = new Director(b)
d.make()
Product1 p = b.getResult()
Director
-builder: Builder
+Director(builder)
+changeBuilder(builder)
+make(type)
creates
retrieves
Builder
+reset()
+buildStepA()
+buildStepB()
+buildStepZ()
builder.reset()
if (type == "simple") {
builder.buildStepA()
} else {
builder.buildStepB()
builder.buildStepZ()
}
ConcreteBuilder1
-result: Product1
+reset()
+buildStepA()
+buildStepB()
+buildStepZ()
+getResult(): Product1
ConcreteBuilder2
-result: Product2
+reset()
+buildStepA()
+buildStepB()
+buildStepZ()
+getResult(): Product2
Product1
Product2
result = new Product2()
result.setFeatureB()
return this.result

Participants

- **Builder** – Interface
  - Specifies an abstract interface for creating parts of a Product object.
  - Declares product construction steps that are common to all types of builders.
- **ConcreteBuilder**
  - Constructs and assembles parts of the product by implementing Builder interface.
  - Provide different implementations of the construction steps.
  - Defines and keeps track of the representation it creates.
  - Provides an interface for retrieving the product.
- **Product**
  - Represents the complex object under construction.
  - Includes classes that define the constituent parts, including interfaces for assembling the parts into the final result.
- **Director**
  - Constructs an object using the Builder interface.
  - The director class defines the order in which to execute the building steps (a series of calls to the builder steps). The builder provides the implementation for those steps.

Documenting the Builder Pattern – SENG301 Style

GoF Term	Our Design
Builder	
ConcreteBuilder	
Director	
Product	
...	

Table: Documenting the Builder Pattern

Pros and Cons

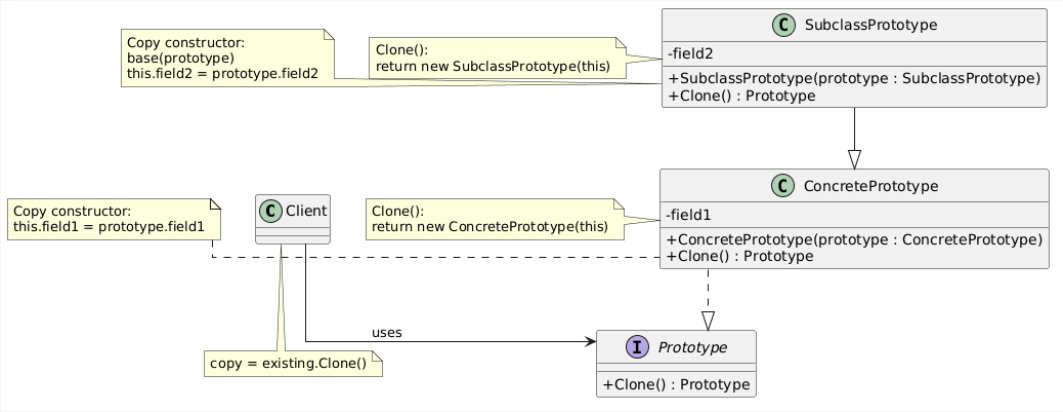
- **Pros**
  - Adheres to the Single Responsibility Principle – Can isolate complex construction code from the business logic of the product.
  - Since it isolates construction and representation, it improves modularity. Each builder contains all code to create and assemble a particular kind of product. The code is written once and could be reused by different directors.
  - Finer control over the construction process – Allows objects to be constructed in a multi-step process.
  - Lets you vary the internal representation of a product – all you have to do is define a new kind of builder.
- **Cons**
  - Overall complexity of the code increases since the pattern requires creating multiple new classes.
  - The client may still need to know which builder to choose, unless this is hidden by a director.

The Prototype Pattern

The Prototype Pattern

- **Intent:** This pattern lets you copy or clone existing objects without making your code dependent on their classes.
- **Problem/ Motivation:** When you want to create an exact copy of an object. But sometimes you only know the interface that the object follows, and not its concrete class.
- **Solution:** Clone a prototype instead of constructing a new object from scratch. The pattern delegates the cloning process to the objects that are being cloned. Declares a common interface which contains clone method.

Structure

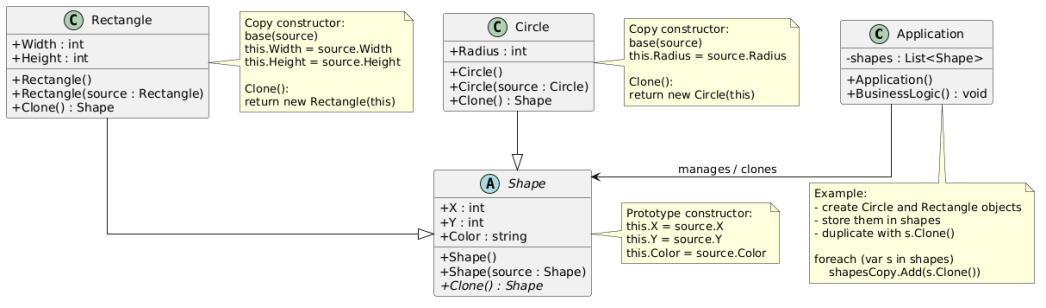


Note: If you extend a Prototype you should override the clone method

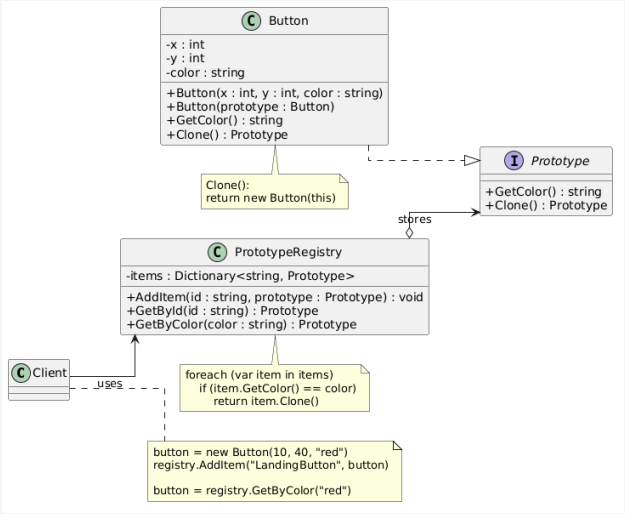
Participants

- **Prototype**
  - The Prototype interface declares the clone method.
- **Concrete Prototype**
  - Implements the cloning method(s) defined by the Prototype interface
  - Copies the original object's data to the clone.
- **Client**
  - Produces a copy of any object that follows the prototype interface.
- **Prototype Registry (optional)**
  - Maintains a registry to access frequently-used prototypes, i.e., stores a set of pre-built objects that are ready to be copied.

Shapes Example



GUI Buttons Example with Prototype Registry



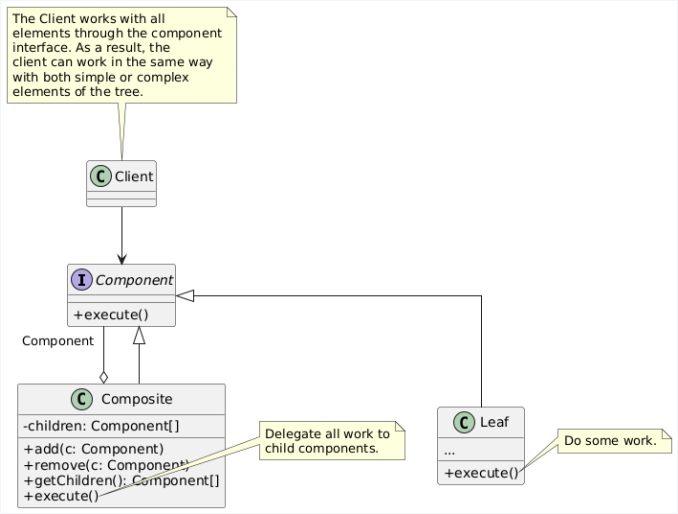
Pros and Cons

- **Pros**
  - Clone objects without coupling to their concrete classes.
  - Produce complex objects easily.
- **Cons**
  - Complex objects that have circular references might be tricky to clone.

The Composite Pattern

- **Intent:** Compose objects into tree structures to represent part-whole hierarchies. Allows treating individual objects and compositions of objects uniformly.
- **Problem/ Motivation:** When we want the client code to treat the composite objects and the atomic objects uniformly. This pattern makes sense only when the core model of your app can be represented as a tree structure. Examples: UI, File system, Graphics editor
- **Solution:** Work with composite and leaf objects through a common interface (e.g., IComponent). **Alternatively,** create an abstract base class that represents both composite and atomic objects.

Structure


The Client works with all elements through the component interface. As a result, the client can work in the same way with both simple or complex elements of the tree.
Client
Component
+execute()
Component
Composite
-children: Component[]
+add(c: Component)
+remove(c: Component)
+getChildren(): Component[]
+execute()
Delegate all work to child components.
Leaf
...
+execute()
Do some work.

Participants

- **Component**
  - The Component defines operations common to both leaves and composites. This may be an interface, or an abstract base class if shared default behaviour is needed.
- **Composite**
  - Defines behaviour for components that have children
  - Stores child components
  - Works with all sub-elements only via the Component interface, doesn't know the concrete classes of its children
- **Leaf**
  - Represents a leaf object in the composition. A Leaf has no children. It is a basic element of the tree.
  - Defines behaviour for primitive objects in the composition.
  - Does the real work, since leaves don't have anyone to delegate the work to.
- **Client**
  - Manipulates the objects in the composition through the Component.

Documenting the Composite Pattern – SENG301 Style (Gift Box Example)

GoF Term	Our Design (Gift Delivery)
Component	`IShipmentItem`
Composite	`LargeBox`, `MediumBox`
Leaf	`Gift`
Client	`CheckoutService`
Operation	`GetTotalPrice()`

Table: Composite Pattern in a Gift Delivery Example

Pros and Cons

- **Pros**
  - Allows having composite and primitive objects. Primitive objects can be composed into more complex objects, which in turn can be composed recursively.
  - Makes the client code simple. The client doesn't know whether they are dealing with a composite or a leaf.
  - Makes it easy to add new kinds of components, i.e., client doesn't have to be changed to add new Component classes
- **Cons**
  - Disadvantage of making it easy to add new components – harder to restrict the components of a composite, i.e., the system cannot enforce a constraint on it. So you'll have to do runtime checks instead.

The Decorator Pattern

The Decorator Pattern (aka Wrapper)

- **Intent:** Attach additional responsibilities to an object dynamically and transparently.
- **Problem/ Motivation:** Overcome problems with Inheritance hierarchies. It is static and you can't alter the behavior of an existing object at runtime. The sealed keyword (*final* in Java) is in use but you would like to add or modify behavior.

The Decorator Pattern (aka Wrapper) contd.

- **Solution:** Enclose a component in another object. The enclosing object is called the ‘decorator’. Easily substitute a “decorator” object with another. Simply compose with the decorators to produce the desired result.
  The decorator conforms to the interface of the component it decorates.

Participants contd.

- **Base Decorator (Base Wrapper)**
  - Maintains a reference to a Component (decorated or wrapped object). The reference type should be declared as the Component interface so it can contain both concrete components and decorators.
  - The base decorator delegates all operations to the decorated or wrapped object.
- **Concrete Decorator (Concrete Wrapper)**
  - Defines extra behaviors that can be added to components dynamically. Overrides methods of the base decorator and executes its own behaviour either before or after calling the base method.

Documenting the Decorator Pattern – SENG301 Style (UI TextView Example)

GoF Term	Our Design (UI Rendering)
Component	`IVisualComponent`
ConcreteComponent	`TextView`
Decorator	`VisualComponentDecorator`
ConcreteDecorator	`BorderDecorator`, `ScrollBarDecorator`
Client	`UIRenderer`

Table: Decorator Pattern in a TextView UI Framework

Pros and Cons

- **Pros**
  - More flexibility than static inheritance – a more flexible way to add responsibilities, i.e., responsibilities could be added or removed dynamically at runtime simply by attaching or detaching them.
  - Can combine several behaviors by wrapping an object in multiple decorators.
  - Composition over inheritance is the key principle
- **Cons**
  - Lots of little objects – this could make the application hard to understand and debug
  - It may become difficult to remove a specific wrapper from the wrappers stack.

The Command Pattern

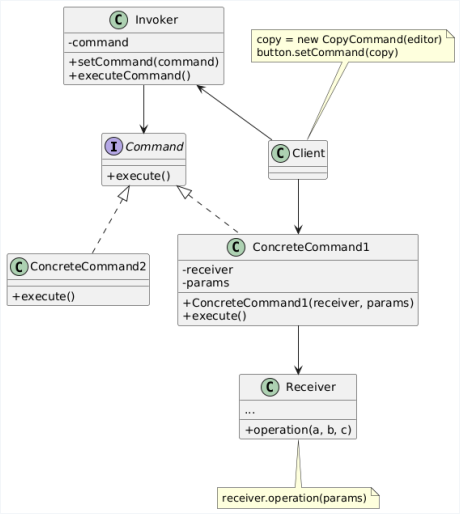
The Command Pattern (aka Transaction)

- **Intent:** Turns a request into a stand-alone object that contains all information about the request. Allows passing requests as method arguments, allows delaying or queuing the execution of a request, and supports undoable operations.
- **Problem/ Motivation:** Text-editor app. You want to have various types of Buttons, e.g., OK Button, Cancel Button, Save Button, Open Button. Additionally, you might want to invoke commands from various places, for example, the Copy function – click a “Copy” button on the toolbar, or copy something via the context menu, or just hit Ctrl+C on the keyboard.

Command Pattern contd.

- **Solution:** separation of concerns – separate the GUI and the business logic.
  Extract all of the request details (e.g., name of the object being called, method name, list of arguments) into a separate command class with a single method that triggers this request.
  A Command object serves as a link between various GUI and business logic objects.
  Make each command implement a common interface, for example `ICommand`, with a method such as `Execute()` that takes no parameters. Any data needed by the command can be stored in the concrete command object, often through its constructor.
- Commands become a convenient middle layer that **reduces coupling** between the GUI and business logic layers.

Structure


Invoker
-command
+setCommand(command)
+executeCommand()
copy = new CopyCommand(editor)
button.setCommand(copy)
Command
+execute()
Client
ConcreteCommand2
+execute()
ConcreteCommand1
-receiver
-params
+ConcreteCommand1(receiver, params)
+execute()
Receiver
...
+operation(a, b, c)
receiver.operation(params)

Participants

- **Invoker**
  - The Sender class (aka invoker) is responsible for initiating requests. – It asks the Command to carry out the request.
  - Stores a reference to a command object.
  - A command may be associated with one or multiple senders (or Invokers).
- **Command**
  - The Command interface usually declares just a single method for executing the command.
- **ConcreteCommand**
  - Implements requests. Implements execute() by invoking the corresponding operations on the Receiver.
  - Parameters required to execute a method on a receiving object can be declared as fields in the concrete command.
  - Command objects could be made immutable by only allowing the initialization of these fields via the constructor.

Participants contd.

- **Receiver**
  - The Receiver class contains the business logic. It knows how to perform the operations associated with carrying out a request.– It does the actual work!
- **Client**
  - Creates and configures a ConcreteCommand object.
  - The client must pass all of the request parameters, including a receiver instance, into the command's constructor.

Pros and Cons

- **Pros**
  - Adheres to the Single Responsibility Principle: Decouples classes that invoke operations from classes that perform these operations, e.g., separating GUI from the business logic.
  - Adheres to the Open/Closed Principle: Can introduce new commands into the app without breaking existing client code.
  - Can implement deferred execution of operations.
- **Cons**
  - The code may become more complicated due to introducing a whole new layer between senders and receivers.

The Memento Pattern

The Memento Pattern

- **Intent:** Capture and externalize an object's internal state (without violating encapsulation) so that the object can be restored to this state later. In other words, save and restore the previous state of an object without revealing the details of its implementation.
- **Problem/ Motivation:** Text editor - undo operations carried out on the text. Avoid bad encapsulation – other objects trying to copy the editor's state from outside. Another example - saving the progress in a game. The "Save progress" function needs to be designed to return a player to the last successful level.

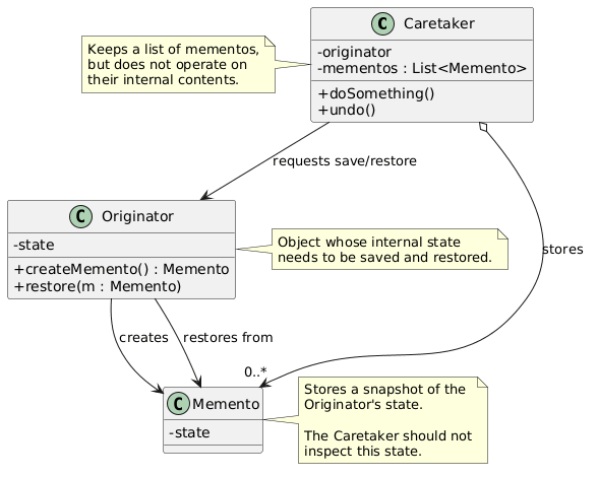
The Memento Pattern contd.

- **Solution:** Memento pattern delegates creating the state snapshots to the actual owner of that state, the originator object. The editor class itself can make the snapshot since it has full access to its own state. The pattern stores a copy of the object's state in a special object called 'memento', and the contents of the memento aren't accessible to any other object except the one that produced it.

  A memento(s) could be stored in a caretaker class (e.g., History class), which works with the memento only via a limited interface that may allow fetching the snapshot's metadata (creation time, the name of the performed operation, etc.), but not the original object's state contained in the snapshot.

  The originator has full access to the memento, the caretaker can only access the metadata.

Structure


Caretaker
-originator
-mementos : List<Memento>
+doSomething()
+undo()
Keeps a list of mementos, but does not operate on their internal contents.
requests save/restore
stores
Originator
-state
+createMemento() : Memento
+restore(m : Memento)
Object whose internal state needs to be saved and restored.
creates
restores from
0..*
Memento
-state
Stores a snapshot of the Originator's state.
The Caretaker should not inspect this state.

Participants

- **Originator**
  - The Class whose state needs to be saved/restored.
  - Creates a Memento which contains its current state. Uses the Memento to restore its internal state.
- **Memento**
  - Snapshot of Originator object's state
  - Stores the internal state of the Originator. Protects from outsiders, allows access by the Originator.
- **Caretaker**
  - Responsible for the Memento's safekeeping.
  - Doesn't operate on the contents of a Memento.
  - Knows when to request a snapshot to be created or restored.
  - Might maintain history of Mementos.

Documenting the Memento Pattern, SENG301 Style

GoF Element	Our Design
Originator	
Memento	
Caretaker	
createMemento()	
restoreMemento(memento)	

Table: Mapping between GoF Memento Pattern and Our Design

Pros and Cons

- **Pros**
  - Produce snapshots of the object's state without violating its encapsulation – i.e., preserves encapsulation boundaries
  - Simplify the originator's code by letting the caretaker maintain the history of the originator's state.
- **Cons**
  - Using Mementos could be expensive (overhead)
  - Hidden costs in caring for Mementos (Caretaker has no idea how much state is in the Memento)

The Strategy Pattern

The Strategy Pattern

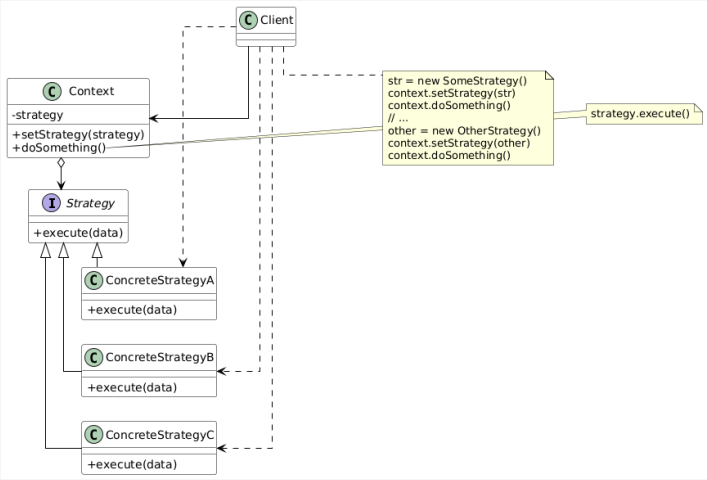
- **Intent:** Define a family of algorithms, encapsulate each algorithm in a separate class, and make the instances interchangeable. With the use of Strategy, the algorithm can vary independently from the clients that use it.
- **Problem/ Motivation:** Change an object's algorithm dynamically, rather than through inheritance.
  Example: Route planning in a Navigation App – uses car routes, cycle routes, walking routes, public transport, etc.

The Strategy Pattern contd.

- **Solution:** Move the algorithms into their own classes – extract all of the different algorithms into separate classes called strategies. e.g., walking strategy, cycling strategy, public transport strategy. The main navigator class (i.e., Context) can switch the transportation strategies based on a button click.

  The context delegates the work to a linked strategy object instead of executing it on its own. The context works with a generic interface which exposes a method for triggering the algorithm encapsulated within the chosen strategy.

Structure


Client
Context
-strategy
+setStrategy(strategy)
+doSomething()
str = new SomeStrategy()
context.setStrategy(str)
context.doSomething()
// ...
other = new OtherStrategy()
context.setStrategy(other)
context.doSomething()
strategy.execute()
Strategy
+execute(data)
ConcreteStrategyA
+execute(data)
ConcreteStrategyB
+execute(data)
ConcreteStrategyC
+execute(data)

Participants

- **Context**
  - Maintains a reference to a Strategy object and communicates via the Strategy interface
  - Configured with a ConcreteStrategy object
  - Uses the Strategy interface to call the algorithm defined by a ConcreteStrategy
- **Strategy**
  - Declares an interface common to all supported algorithms
  - Declares a method the Context uses to execute a specific strategy
- **ConcreteStrategy**
  - Implements a specific algorithm using the Strategy interface

Group Exercise - Identify Strategy Participants

- Identify the participants of the Strategy pattern when implementing an e-commerce application. Assume that the application needs to support various payment methods such as credit card or Paypal.
- Use the template on the next slide to document the participants of the pattern.
- Hint: Think about how each payment method validates payment details and processes the payment, while the checkout code works through a common payment interface.

Documenting The Strategy Pattern – SENG301 Style

GoF Term	Our Design
Context	
Strategy	
ConcreteStrategyA	
ConcreteStrategyB	
ConcreteStrategyC	
Algorithm()	

Table: Mapping between GoF Strategy Pattern and Our Design

Pros and Cons

- **Pros**
  - Adheres to Open/Closed Principle: You can introduce new strategies without having to change the Context
  - Isolates the implementation details of an algorithm from the code that uses it
  - An alternative to subclassing (Composition over inheritance) – allows swapping algorithms at runtime
  - Eliminates the need for conditional statements to select desired behavior
  - Client can select different implementations of the strategies with different trade-offs
- **Cons**
  - Client must be aware of the different strategies before it can select an appropriate one
  - Communication overhead between Strategy and Context
  - Might complicate the code when you have only a few algorithms and they rarely change

The Template Method Pattern

The Template Method Pattern

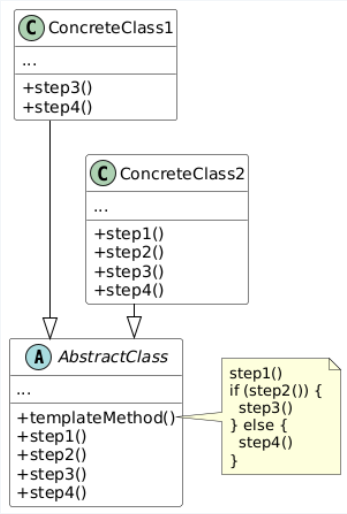
- **Intent:** Define the skeleton of an algorithm, deferring some operations to subclasses. Allows the subclasses to redefine steps of the algorithm without changing the algorithm's structure.
- **Problem/ Motivation:** When you want to let clients extend only particular steps of an algorithm, but not the whole algorithm or its structure

The Template Method Pattern contd.

- **Solution:** Put the skeleton in an abstract superclass and use subclass operations to provide the details – Break down an algorithm into a series of steps, turn these steps into methods, and put a series of calls to these methods inside a single template method. The steps may either be abstract, or have some default implementation.

  Template Method can be easily recognized if you see a method in base class that calls a set of other methods that are either abstract or empty.

Structure


ConcreteClass1
...
+step3()
+step4()
ConcreteClass2
...
+step1()
+step2()
+step3()
+step4()
AbstractClass
...
+templateMethod()
+step1()
+step2()
+step3()
+step4()
step1()
if (step2()) {
step3()
} else {
step4()
}

Participants

- **AbstractClass**
  - Implements a template method defining the skeleton of an algorithm (i.e., steps of an algorithm)
  - Defines abstract primitive operations, e.g., step1(), step2()
- **ConcreteClass**
  - Implements primitive operations (i.e., the required customisable steps) to carry out subclass-specific steps of the algorithm
  - Can override all of the steps but not the template method

Documenting The Template Method Pattern – SENG301 Style

GoF Term	Our Design
AbstractClass	
ConcreteClassA	
ConcreteClassB	
TemplateMethod()	
Primitive operations	
Hook operations	

Table: Mapping between GoF Template Method Pattern and Our Design

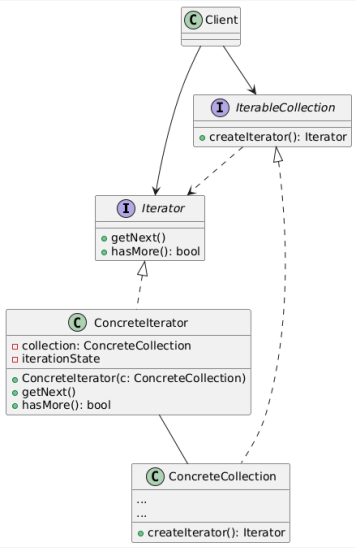
Pros and Cons

- **Pros**
  - A fundamental technique for code reuse
  - hook operations – provide default behaviour that subclasses can extend if necessary. A hook operation often does nothing by default in the parent class.
- **Cons**
  - You might violate the Liskov Substitution Principle by suppressing a default step implementation via a subclass.
  - Harder to maintain the more steps you have.

The Iterator Pattern

- **Intent:** Provide a way to access or traverse the elements of an aggregate object sequentially without exposing its underlying representation (or internal data structure, e.g., list, stack, tree, graph)
- **Problem/ Motivation:** You might want to traverse a list or a tree in different ways (e.g., depth-first or breadth-first)
- **Solution:** Take the responsibility/ mechanism of access and traversal out of the list object and put it into an 'iterator' object so that you can define different iterators for different traversal scenarios
- You can have multiple iterators, all iterators must implement the same interface

Structure


Client
IterableCollection
+ createIterator(): Iterator
Iterator
+ getNext()
+ hasMore(): bool
ConcreteIterator
- collection: ConcreteCollection
- iterationState
+ ConcreteIterator(c: ConcreteCollection)
+ getNext()
+ hasMore(): bool
ConcreteCollection
...
...
+ createIterator(): Iterator

Participants

- **Iterator Interface**
  - Declares operations required for traversing a collection, e.g., get next element, get current position
- **Collection Interface**
  - Declares methods for creating iterators compatible with the collection
- **Concrete Iterator**
  - Specific implementation
- **Concrete Collection**
  - Returns a new instance of a particular concrete iterator class
- **Client**
  - Works with both collections and iterators via their interfaces
  - Does not know about the specific implementations

Pros and Cons

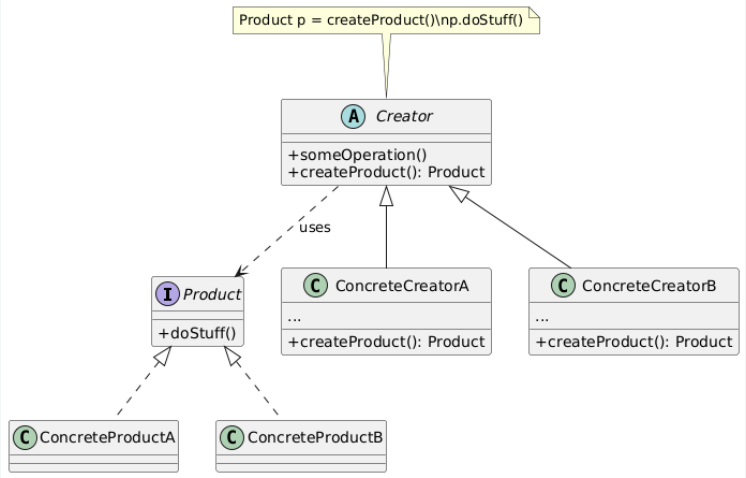
- **Pros**
  - Adheres to Single Responsibility Principle – have different traversal algorithms in separate classes
  - Adheres to Open/Closed Principle – allows implementing new types of collections and iterators without breaking anything
  - Can iterate over the same collection in parallel because each iterator object contains its own iteration state
- **Cons**
  - Less efficient in some cases, e.g., specialised collections
  - Could be too complex when dealing with simple collections

The Factory Method Pattern

The Factory Method Pattern

- **Intent:** Lets a class defer instantiation to subclasses. Define an interface for creating an object but let subclasses decide which class to instantiate.
- **Problem/ Motivation:** Use Factory Method when a class defines a general workflow but should let subclasses decide which concrete product to create. or when you want to provide users of your library or framework with a way to extend its internal components.
- **Solution:** Replaces direct object construction calls (which use the new operator) with calls to a special factory method.
  The objects are still created via the new operator, but it's being called from within the factory method. Now you can override the factory method in a subclass and change the class of products being created by the method.

Structure


Product p = createProduct()\np.doStuff()
Creator
+someOperation()
+createProduct(): Product
uses
Product
+doStuff()
ConcreteCreatorA
...
+createProduct(): Product
ConcreteCreatorB
...
+createProduct(): Product
ConcreteProductA
ConcreteProductB

Participants

- **Creator Class**
  - Declares the factory method that returns new product objects – return type should match the product interface
  - Contains core business logic related to products
- **Concrete Creator**
  - Overrides the factory method and returns a different type of product
- **Product Interface**
  - Common to all products that can be produced by the creator and its subclasses
- **Concrete Product**
  - Different implementations of the product interface

The Factory Method Pattern contd.

- You can declare the factory method (e.g., createProduct()) as abstract to force all subclasses to implement their own versions of the method
- Products need to have a common base class or interface. The factory method should have its return type declared as this interface

Documenting The Factory Method – SENG301 Style

GoF Term	Our Design
Abstract Creator	Player
Concrete Creator	Warrior
Concrete Creator	Wizard
Abstract Product	Weapon
Concrete Product	Sword
Concrete Product	Wand
Factory Method	`makeWeapon()`

Table: Mapping between GoF Factory Method Pattern and Our Design

Pros and Cons

- **Pros**
  - The code only deals with the Product Interface; therefore, it can work with any user-defined ConcreteProduct classes
  - Adheres to the Single Responsibility Principle - product creation is handled by one place in the program
  - Adheres to the Open/Closed Principle - you can introduce new types of products without breaking existing client code
  - Can have parallel hierarchies – creators & products that can have interactions between them
- **Cons**
  - Clients have to subclass the Creator class just to create a particular ConcreteProduct object, i.e., the code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern.

The Abstract Factory Pattern

The Abstract Factory Pattern

- **Intent:** Provide an interface for creating families of related or dependent objects without specifying their concrete classes. In other words, it is similar to the factory method, but we want to create whole families of compatible products.
- **Problem/ Motivation:** You don't want to change existing code when adding new products or families of products to the program.

  Imagine that you're creating a furniture shop simulator. You need to create sets (or families) of furniture.

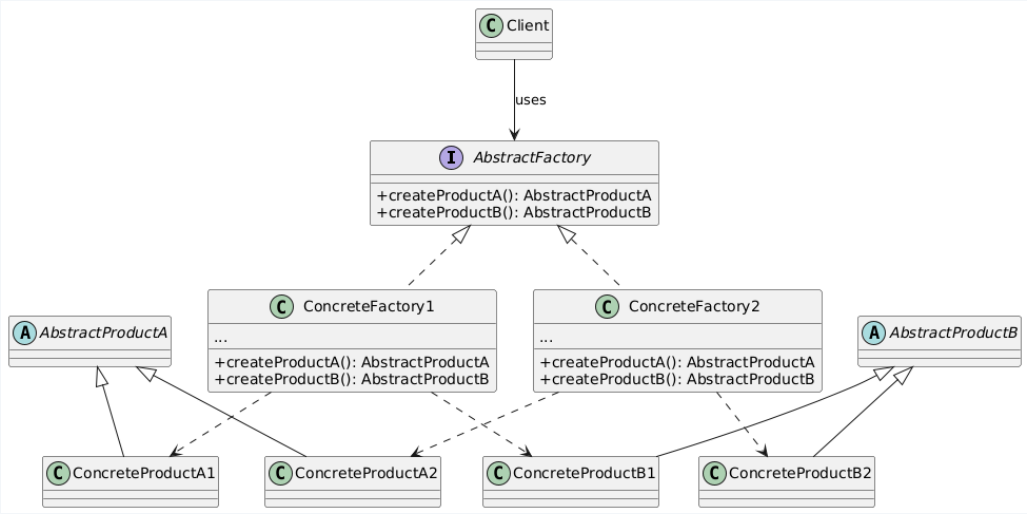
The Abstract Factory Pattern contd.

- **Solution:**
  Explicitly declare interfaces (i.e., abstract product types) for each distinct product of the product family.

  Make all variants of products extend or implement those interfaces.

  Create an AbstractFactory interface which contains a list of creation methods for all products that are part of the product family. – These methods must return the abstract product types.

Structure


Client
uses
AbstractFactory
+createProductA(): AbstractProductA
+createProductB(): AbstractProductB
ConcreteFactory1
...
+createProductA(): AbstractProductA
+createProductB(): AbstractProductB
ConcreteFactory2
...
+createProductA(): AbstractProductA
+createProductB(): AbstractProductB
AbstractProductA
AbstractProductB
ConcreteProductA1
ConcreteProductA2
ConcreteProductB1
ConcreteProductB2

Participants

- **Abstract Factory**
  - Declares a set of methods for creating each of the abstract products
- **Concrete Factory**
  - Implements the creation methods of the abstract factory
- **Abstract Product(s)**
  - Interfaces declared for distinct but related products
- **Concrete Product(s)**
  - Various implementations of abstract products

Documenting The Abstract Factory Pattern – SENG301 Style

GoF Term	Our Design
AbstractFactory	
ConcreteFactory	
AbstractProduct	
ConcreteProduct	
Client	
Factory Methods	

Table: Mapping between GoF AbstractFactory Pattern and Our Design

Pros and Cons

- **Pros**
  - Adheres to Single Responsibility Principle - product creation code in one place
  - Adheres to Open/Closed Principle - You can introduce new variants of products without breaking existing client code
  - Ensure the products you're getting from a factory are compatible with each other
  - Avoids tight coupling between concrete products and client code
- **Cons**
  - Code can become more complicated than it should be since a lot of new interfaces and classes are introduced